

bok25

基礎

課、計算機網絡



2025/10/10 10:10:10





UDP的概述



IP数据报服务

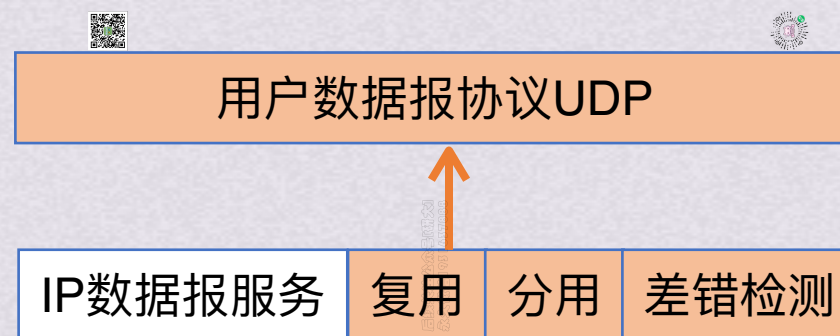
复用

分用

差错检测



UDP的概述





UDP的概述

用户数据报协议UDP

特点： 1.UDP是无连接的，发送数据之前不需要建立连接



2.UDP使用尽最大努力交付，不保证可靠交付

3.UDP是面向报文的



发送方的UDP对应用程序交下来的报文，不合并也不拆分，保留报文的边界，在添加首部后就向下交付IP层应用层给多长的报文，UDP就照样发送，也就是说，UDP一次交付一个完整的报文所以要求应用程序选择合适大小的报文，如果太长，UDP把它交给IP层后，IP层在传送时可能进行分片如果太短，UDP把它交给IP层后，会使IP数据报的首部相对长度太大

4.UDP没有拥塞控制因此，网络拥塞不会使源主机发送速率降低

5.UDP支持一对一、一对多、多对一和多对多的互通信

6.UDP的首部开销小，只有8字节



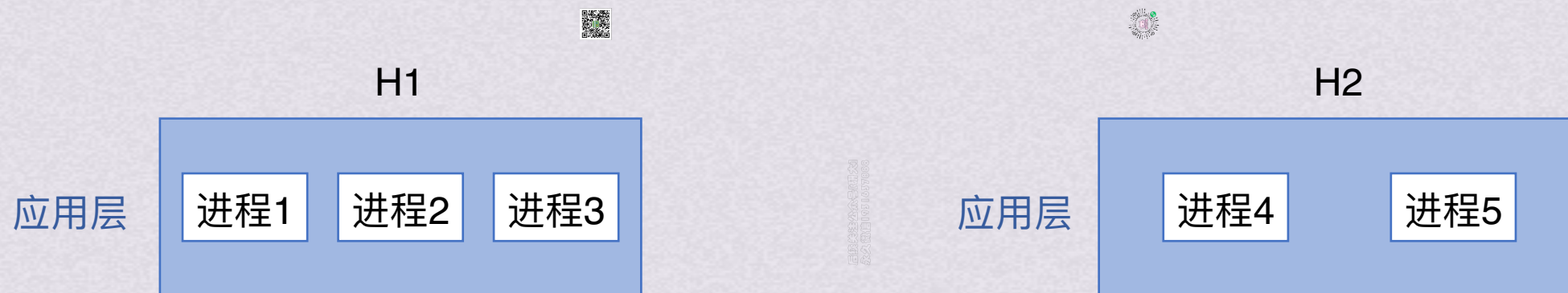
UDP的概述



UDP的复用和分用



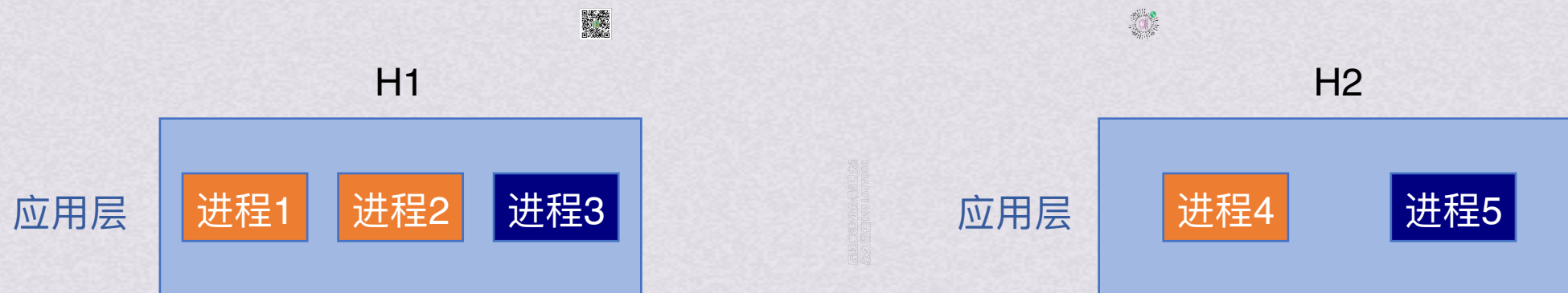
UDP的复用和分用



进程1需要和进程4通信
进程2需要和进程4通信
进程3需要和进程5通信



UDP的复用和分用



进程1需要和进程4通信
进程2需要和进程4通信
进程3需要和进程5通信



UDP的复用和分用



主机H1的操作系统为三个进程分别指派了端口

端口号为a,b,c

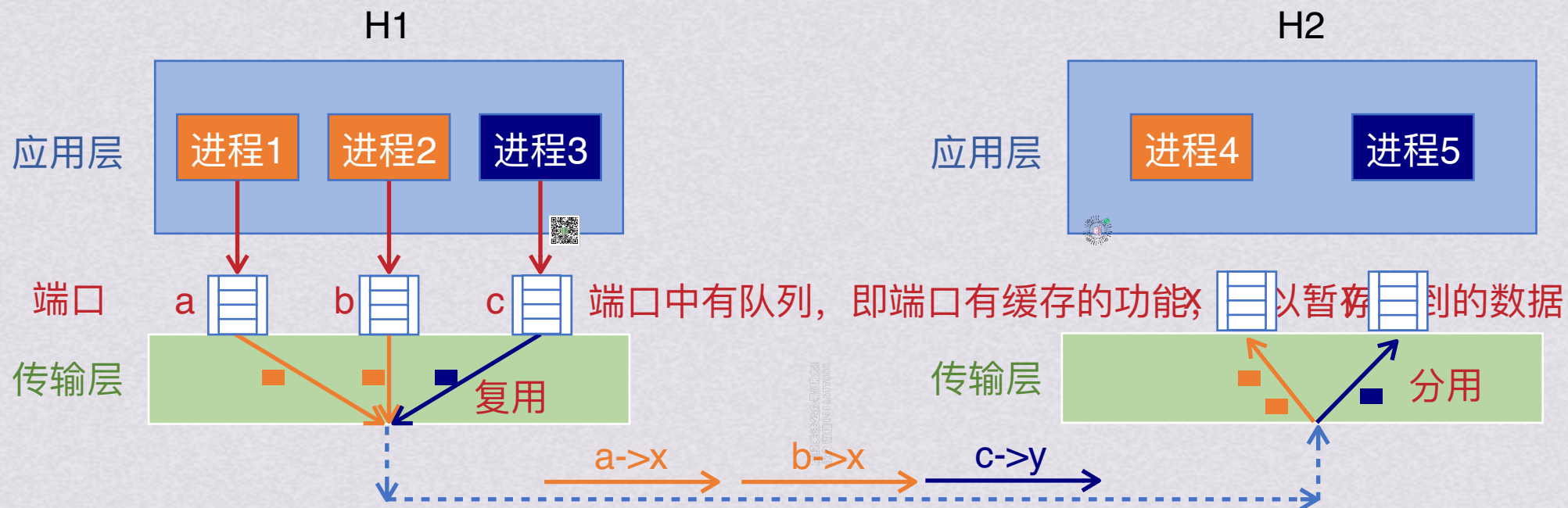
进程1需要和进程4通信

进程2需要和进程4通信

进程3需要和进程5通信



UDP的复用和分用

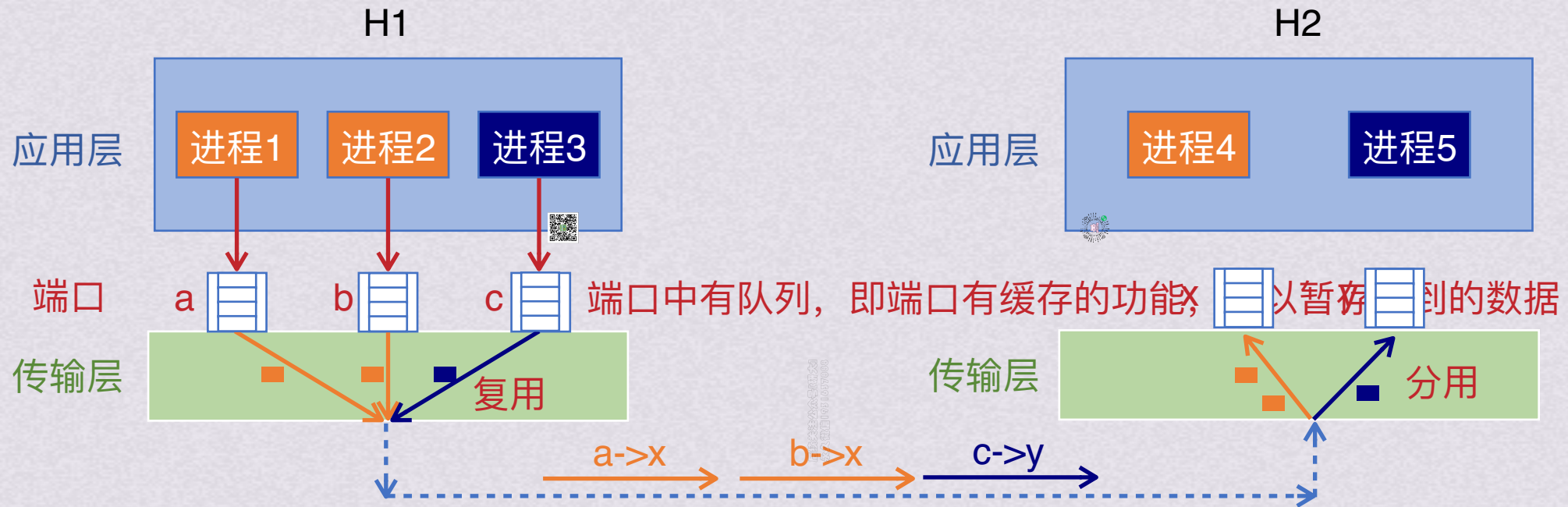


假设H1中进程已经知道了对方进程4和进程5端口号分别为x,y

主机H1的操作系统为三个进程分别指派了端口
端口号为a,b,c

进程1需要和进程4通信
进程2需要和进程4通信
进程3需要和进程5通信

UDP的复用和分用



假设H1中进程已经知道了对方进程4和进程5端口号分别为x,y

主机H1的操作系统为三个进程分别指派了端口

端口号为a,b,c

多路复用/分用

接收端进行多路分用

传输层根据头部信息将收到的Segment交给正确的Socket(不同进程)

发送端进行多路复用

从多个Socket接受数据, 为每块数据封装上头部信息, 生成Segment, 交给网络层



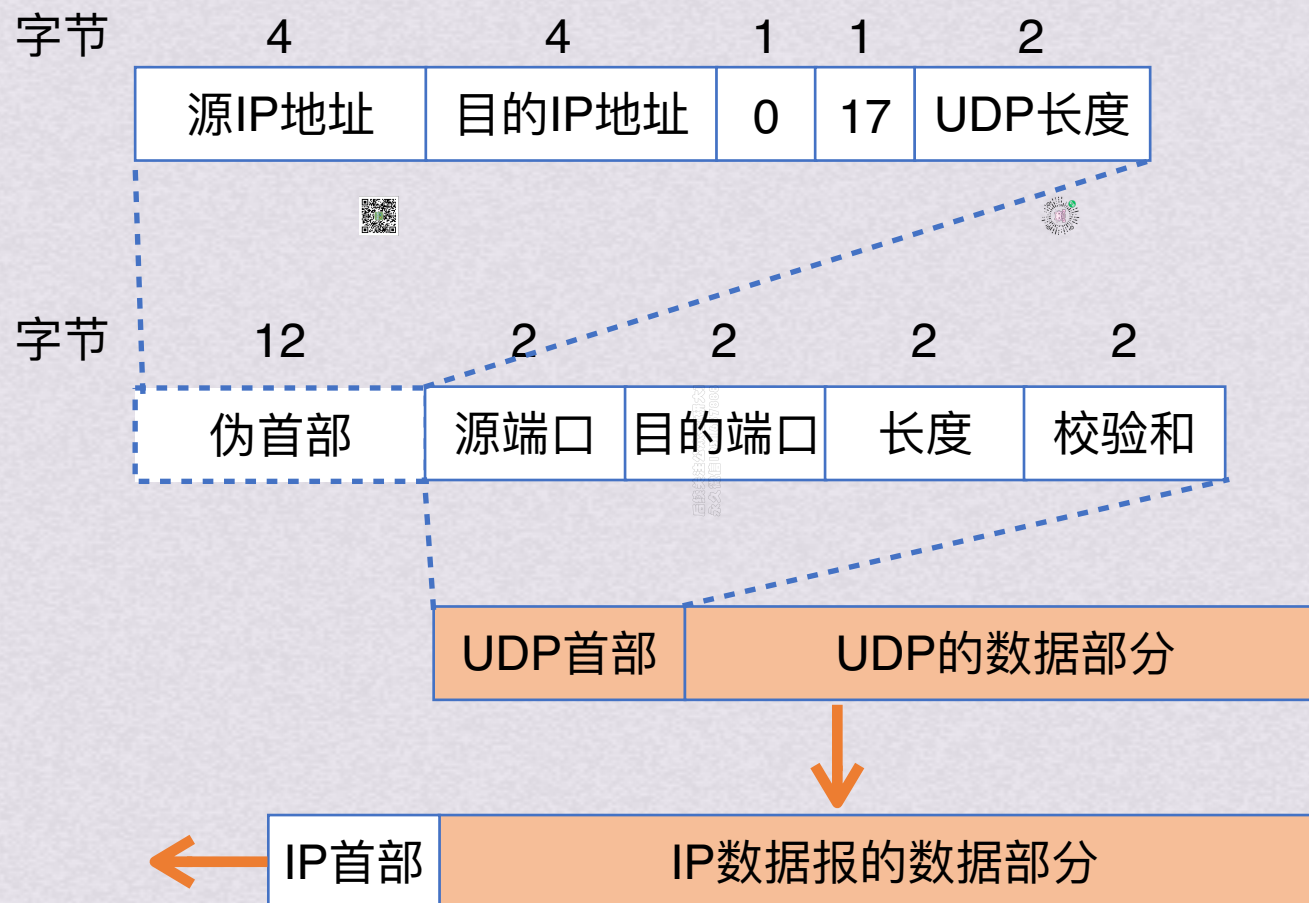
UDP的复用和分用



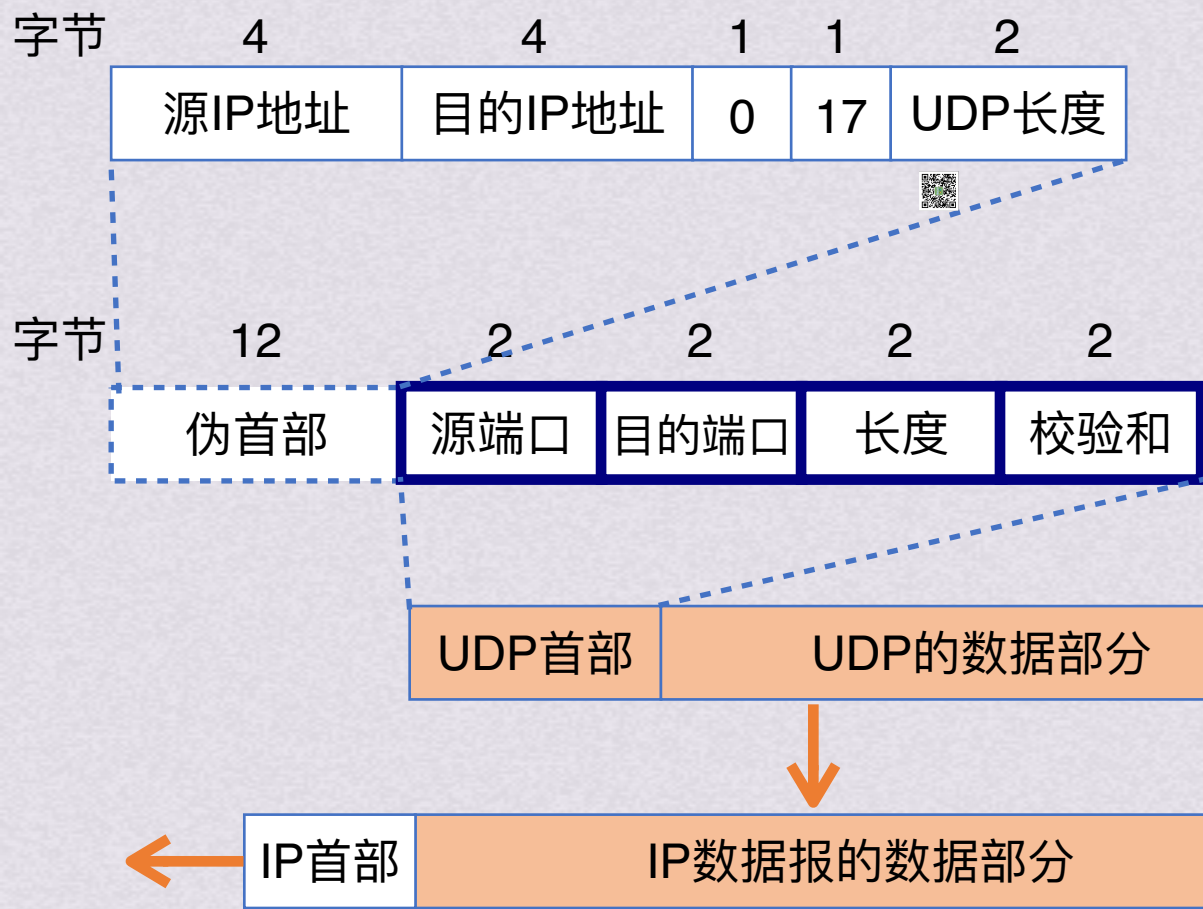
UDP的首部格式



UDP的首部格式



UDP的首部格式



源端口：源端口号，在需要对方回信时选用不需要时可用全0

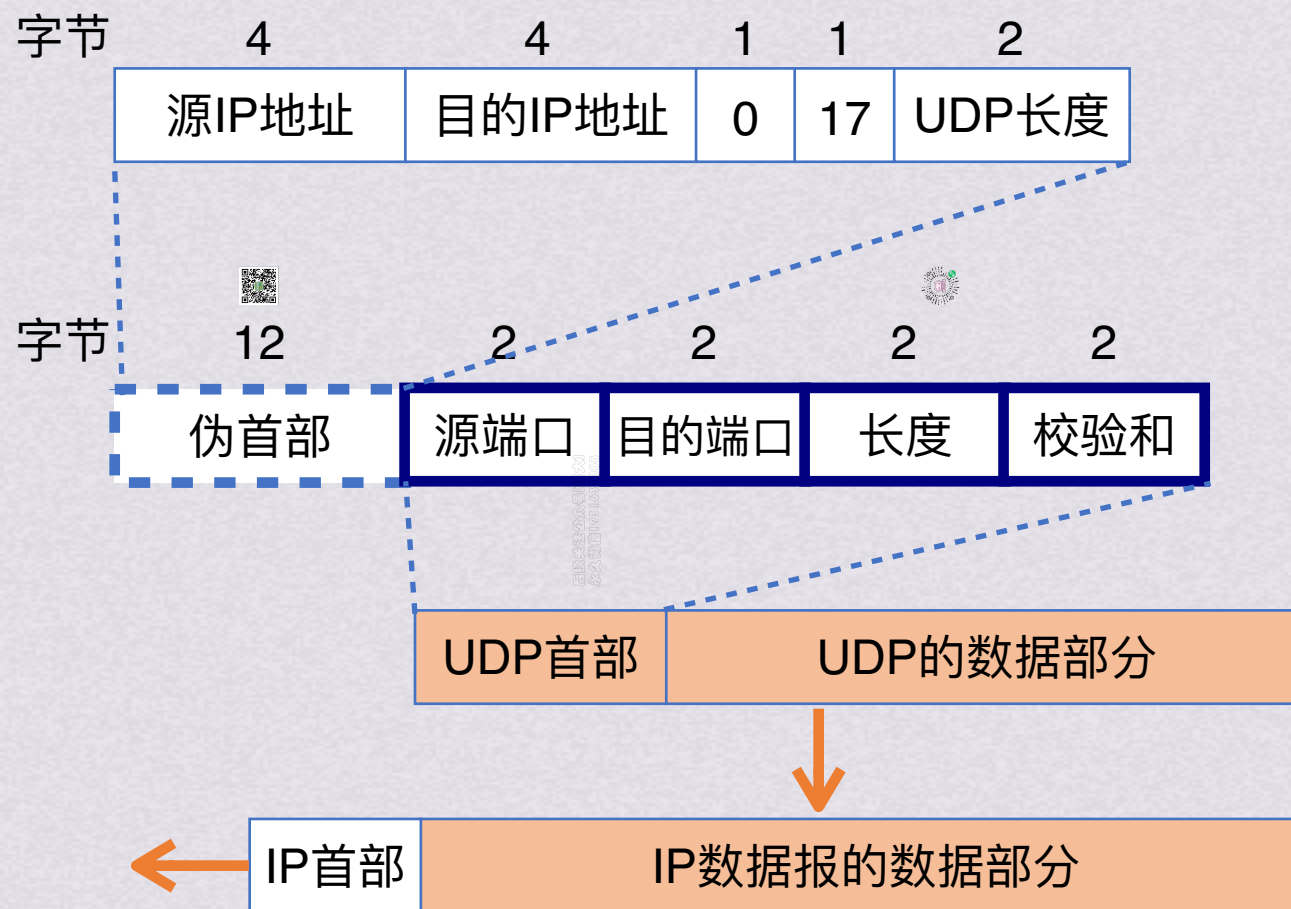
目的端口：目的端口号这在终点交付报文时必须使用
如果接收方发现目的端口号不正确，丢弃该报文，并返回“端口不可达”ICMP差错报文给发送方

长度：UDP用户数据报的长度，其最小值是8（仅有首部）

校验和：检测UDP用户数据报在传输中是否有错，有错就丢弃



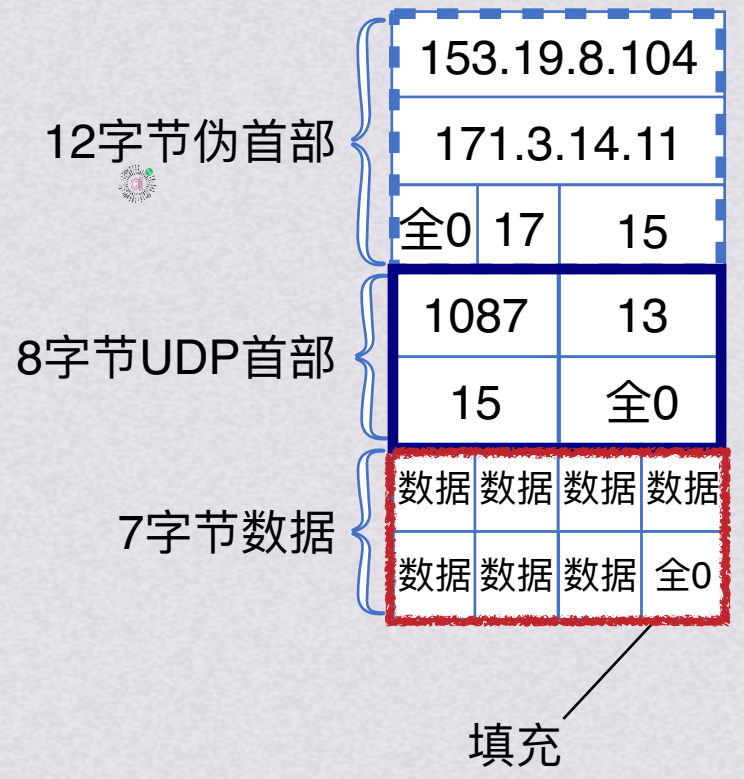
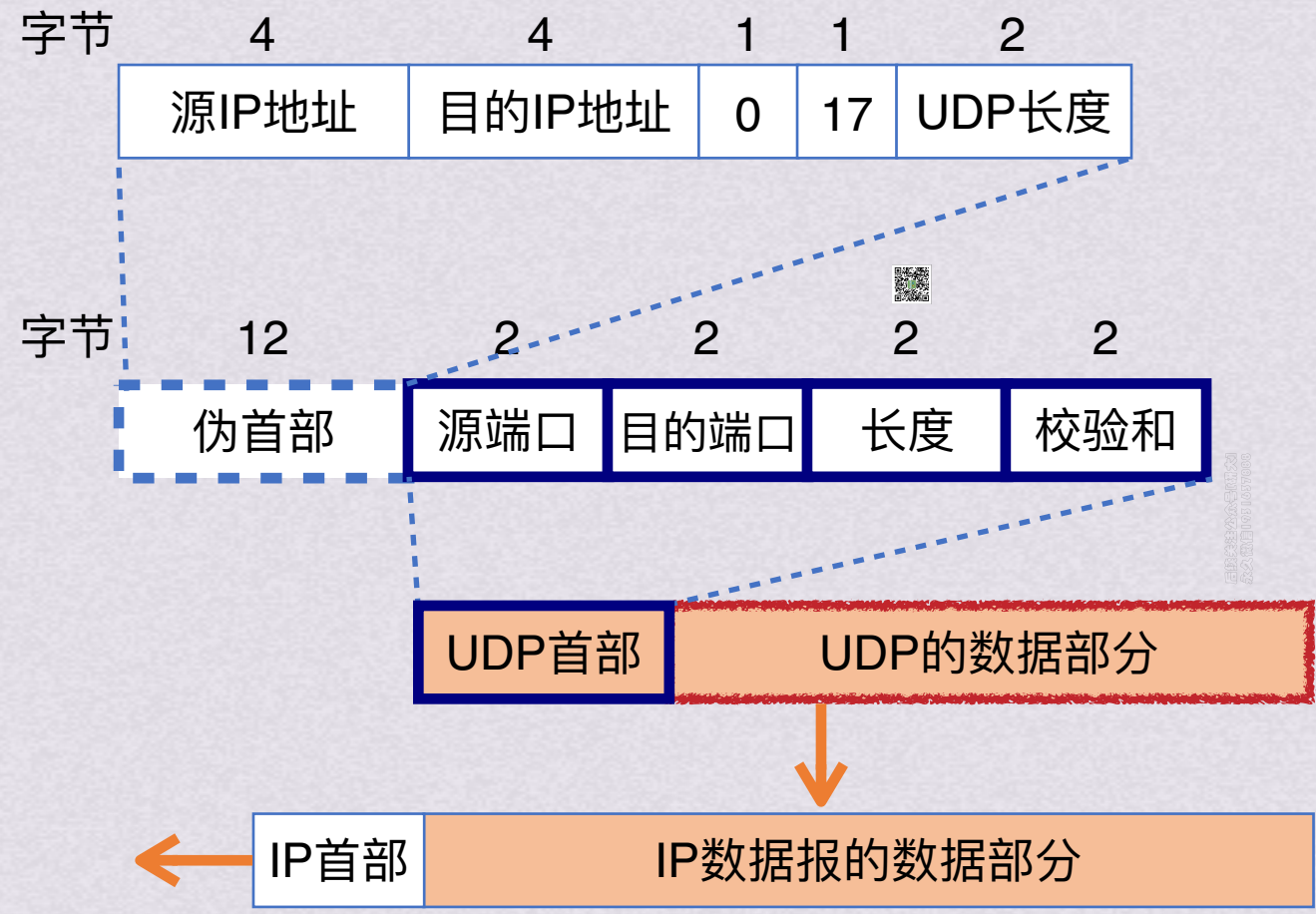
UDP的首部格式



伪首部是指这个首部不属于UDP数据报真正的首部，而只是在计算校验和时临时添加在UDP用户数据报前面伪首部既不向下传送也不向上递交，仅仅是为了计算校验和

UDP把首部和数据部分一起检验，和IP不同（IP的校验和只检测首部）

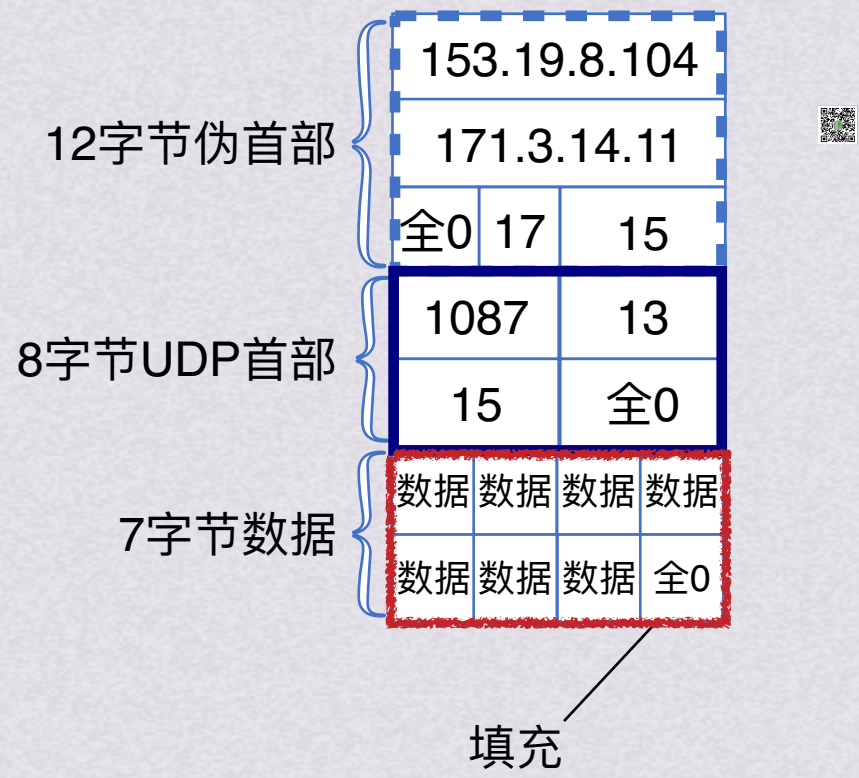
UDP的首部格式



伪首部是指这个首部不属于UDP数据报真正的首部，而只是在计算校验和时临时添加在UDP用户数据报前面伪首部既不向下传送也不向上递交，仅仅是为了计算校验和

UDP把首部和数据部分一起检验，和IP不同（IP的校验和只检测首部）

UDP的首部格式

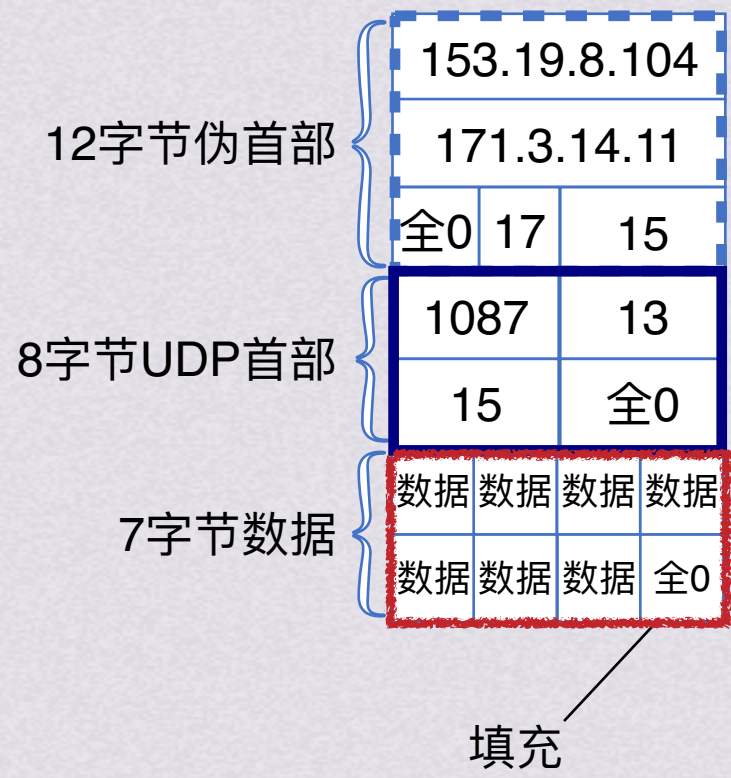


10011001	00010011	→	153.19
00001000	01101000	→	8.104
10101011	00000011	→	171.3
00001110	00001011	→	14.11
00000000	00010001	→	0和17
00000000	00001111	→	15
00000100	00111111	→	1087
00000000	00001101	→	13
00000000	00001111	→	15
00000000	00000000	→	0(检验和)
01010100	01000101	→	数据
01010011	01010100	→	数据
01001001	01001110	→	数据
01000111	00000000	→	数据和0(填充)
10010110	11101101	→	求和得出的结果
01101001	00010010	→	检验和

按二进制反码运算求和
将得出的结果求反码

UDP的首部格式

按二进制反码运算求和 10010110 11101101 → 求和得出的结果
将得出的结果求反码 01101001 00010010 → 检验和

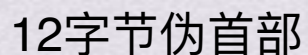


在发送方，计算出这些16位字的和将此和的二进制反码写入校验和字段后，就发送这样的UDP用户数据报

在接收方，把收到的UDP用户数据报连同伪首部以及可能的填充全0字节一起，按二进制反码求这些16位字的和
当无差错时结果应为全1，否则就表明有差错，接收方应丢弃这个UDP用户数据报



10010110 11101101



8字节UDP首部

7字节数据

填充

在发送方，计算出这些16位字的和将此和的二进制反码写入校验和字段后，就发送这样的UDP用户数据报

在接收方，把收到的UDP用户数据报连同伪首部以及可能的填充全0字节一起，按二进制反码求这些16位字的和当无差错时结果应为全1否则就表明有差错，接收方应丢弃这个UDP用户数据报



UDP检验和的计算



UDP检验和的计算



- 1.把所有数据分成16bit的片段
- 2.依次求和，如果最高位有进位，就挪到最低位进行一次+1操作
- 3.最后得到的结果取反码



UDP检验和的计算

1.把所有数据分成16bit的片段

2.依次求和，如果最高位有进位，就挪到最低位进行一次+1操作



3.最后得到的结果取反码



12字节伪首部	153.19.8.104			
	171.3.14.11			
	全0	17	15	
8字节UDP首部	1087		13	
	15		全0	
7字节数据	数据	数据	数据	数据
	数据	数据	数据	全0

10011001	00010011	→	153.19
00001000	01101000	→	8.104
10101011	00000011	→	171.3
00001110	00001011	→	14.11
00000000	00010001	→	0和17
00000000	00001111	→	15
00000100	00111111	→	1087
00000000	00001101	→	13
00000000	00001111	→	15
00000000	00000000	→	0(检验和)
01010100	01000101	→	数据
01010011	01010100	→	数据
01001001	01001110	→	数据
01000111	00000000	→	数据



-



UDP检验和的计算

- 1.把所有数据分成16bit的片段
- 2.依次求和，如果最高位有进位，就挪到最低位进行一次+1操作
- 3.最后得到的结果取反码



000011010000

10011001	00010011	→	153.19
+ 00001000	01101000	→	8.104
10100001	01111011		
+ 10101011	00000011	→	171.3
1 01001100	01111110		

00001110	00001011	→	14.11
00000000	00010001	→	0和17
00000000	00001111	→	15
00000100	00111111	→	1087
00000000	00001101	→	13
00000000	00001111	→	15
00000000	00000000	→	0(检验和)
01010100	01000101	→	数据
01010011	01010100	→	数据
01001001	01001110	→	数据
01000111	00000000	→	数据



UDP检验和的计算

- 1.把所有数据分成16bit的片段
- 2.依次求和，如果最高位有进位，就挪到最低位进行一次+1操作
- 3.最后得到的结果取反码



0000110101010101

$$\begin{array}{r} 10011001 \quad 00010011 \rightarrow 153.19 \\ + 00001000 \quad 01101000 \rightarrow 8.104 \\ \hline 10100001 \quad 01111011 \\ + 10101011 \quad 00000011 \rightarrow 171.3 \\ \hline 01001100 \quad 01111110 \\ + 1 \\ \hline 01001100 \quad 01111111 \end{array}$$

00001110	00001011	→ 14.11
00000000	00010001	→ 0和17
00000000	00001111	→ 15
00000100	00111111	→ 1087
00000000	00001101	→ 13
00000000	00001111	→ 15
00000000	00000000	→ 0(检验和)
01010100	01000101	→ 数据
01010011	01010100	→ 数据
01001001	01001110	→ 数据
01000111	00000000	→ 数据



UDP检验和的计算

- 1.把所有数据分成16bit的片段
- 2.依次求和，如果最高位有进位，就挪到最低位进行一次+1操作
- 3.最后得到的结果取反码



0000100100000000

$$\begin{array}{r} 10011001 \quad 00010011 \rightarrow 153.19 \\ + 00001000 \quad 01101000 \rightarrow 8.104 \\ \hline 10100001 \quad 01111011 \\ + 10101011 \quad 00000011 \rightarrow 171.3 \\ \hline 01001100 \quad 01111110 \\ + 1 \\ \hline 01001100 \quad 01111111 \\ + 00001110 \quad 00001011 \rightarrow 14.11 \\ \hline 01011010 \quad 10001010 \end{array}$$

00000000	00010001	→ 0和17
00000000	00001111	→ 15
00000100	00111111	→ 1087
00000000	00001101	→ 13
00000000	00001111	→ 15
00000000	00000000	→ 0(检验和)
01010100	01000101	→ 数据
01010011	01010100	→ 数据
01001001	01001110	→ 数据
01000111	00000000	→ 数据



UDP检验和的计算

- 1.把所有数据分成16bit的片段
- 2.依次求和，如果最高位有进位，就挪到最低位进行一次+1操作
- 3.最后得到的结果取反码



0000100100000000

10011001	00010011	→ 153.19
+ 00001000	01101000	→ 8.104
10100001	01111011	
+ 10101011	00000011	→ 171.3
01001100	01111110	
+	1	
01001100	01111111	
+ 00001110	00001011	→ 14.11
01011010	10001010	
+ 00000000	00010001	→ 0和17
01011010	10011011	

00000000	00001111	→ 15
00000100	00111111	→ 1087
00000000	00001101	→ 13
00000000	00001111	→ 15
00000000	00000000	→ 0(检验和)
01010100	01000101	→ 数据
01010011	01010100	→ 数据
01001001	01001110	→ 数据
01000111	00000000	→ 数据



UDP检验和的计算

- 把所有数据分成16bit的片段
- 依次求和，如果最高位有进位，就挪到最低位进行一次+1操作
- 最后得到的结果取反码



00000000

00000100	00111111	→ 1087
00000000	00001101	→ 13
00000000	00001111	→ 15
00000000	00000000	→ 0(检验和)
01010100	01000101	→ 数据
01010011	01010100	→ 数据
01001001	01001110	→ 数据
01000111	00000000	→ 数据

10011001	00010011	→ 153.19
+ 00001000	01101000	→ 8.104
10100001	01111011	
+ 10101011	00000011	→ 171.3
01001100	01111110	
+	1	
01001100	01111111	
+ 00001110	00001011	→ 14.11
01011010	10001010	
+ 00000000	00010001	→ 0和17
01011010	10011011	
+ 00000000	00001111	→ 15
01011010	10101010	



UDP检验和的计算

- 1.把所有数据分成16bit的片段
- 2.依次求和，如果最高位有进位，就挪到最低位进行一次+1操作
- 3.最后得到的结果取反码



0000110110110110

00000000	00001101	→ 13
00000000	00001111	→ 15
00000000	00000000	→ 0(检验和)
01010100	01000101	→ 数据
01010011	01010100	→ 数据
01001001	01001110	→ 数据
01000111	00000000	→ 数据

10011001	00010011	→ 153.19
+ 00001000	01101000	→ 8.104
10100001	01111011	
+ 10101011	00000011	→ 171.3
01001100	01111110	
+	1	
01001100	01111111	
+ 00001110	00001011	→ 14.11
01011010	10001010	
+ 00000000	00010001	→ 0和17
01011010	10011011	
+ 00000000	00001111	→ 15
01011010	10101010	
+ 00000100	00111111	→ 1087
01011110	11101001	



UDP检验和的计算

- 把所有数据分成16bit的片段
- 依次求和，如果最高位有进位，就挪到最低位进行一次+1操作
- 最后得到的结果取反码



000010010000

00000000	00001111	→ 15
00000000	00000000	→ 0(检验和)
01010100	01000101	→ 数据
01010011	01010100	→ 数据
01001001	01001110	→ 数据
01000111	00000000	→ 数据

10011001	00010011	→ 153.19
+ 00001000	01101000	→ 8.104
10100001	01111011	
+ 10101011	00000011	→ 171.3
01001100	01111110	
+	1	
01001100	01111111	
+ 00001110	00001011	→ 14.11
01011010	10001010	
+ 00000000	00010001	→ 0和17
01011010	10011011	
+ 00000000	00001111	→ 15
01011010	10101010	
+ 00000100	00111111	→ 1087
01011110	11101001	
+ 00000000	00001101	→ 13
01011110	11110110	



UDP检验和的计算

- 把所有数据分成16bit的片段
- 依次求和，如果最高位有进位，就挪到最低位进行一次+1操作
- 最后得到的结果取反码

01011111 00000101

00000000	00000000	→ 0(检验和)
01010100	01000101	→ 数据
01010011	01010100	→ 数据
01001001	01001110	→ 数据
01000111	00000000	→ 数据

10011001	00010011	→ 153.19
+ 00001000	01101000	→ 8.104
10100001	01111011	
+ 10101011	00000011	→ 171.3
01001100	01111110	
+ 1		
01001100	01111111	
+ 00001110	00001011	→ 14.11
01011010	10001010	
+ 00000000	00010001	→ 0和17
01011010	10011011	
+ 00000000	00001111	→ 15
01011010	10101010	
+ 00000100	00111111	→ 1087
01011110	11101001	
+ 00000000	00001101	→ 13
01011110	11110110	
+ 00000000	00001111	→ 15
01011111	00000101	

写不下了写左边



UDP校验和的计算

- 1.把所有数据分成16bit的片段
- 2.依次求和，如果最高位有进位，就挪到最低位进行一次+1操作
- 3.最后得到的结果取反码

$$\begin{array}{r}
 01011111 \quad 00000101 \\
 + 00000000 \quad 00000000 \\
 \hline
 01011111 \quad 00000101 \rightarrow 0(\text{校验和})
 \end{array}$$

01010100 01000101 → 数据
 01010011 01010100 → 数据
 01001001 01001110 → 数据
 01000111 00000000 → 数据

$$\begin{array}{r}
 10011001 \quad 00010011 \rightarrow 153.19 \\
 + 00001000 \quad 01101000 \rightarrow 8.104 \\
 \hline
 10100001 \quad 01111011 \\
 + 10101011 \quad 00000011 \rightarrow 171.3 \\
 \hline
 01001100 \quad 01111110 \\
 + \quad 1 \\
 \hline
 01001100 \quad 01111111 \\
 + 00001110 \quad 00001011 \rightarrow 14.11 \\
 \hline
 01011010 \quad 10001010 \\
 + 00000000 \quad 00010001 \rightarrow 0和17 \\
 \hline
 01011010 \quad 10011011 \\
 + 00000000 \quad 00001111 \rightarrow 15 \\
 \hline
 01011010 \quad 10101010 \\
 + 00000100 \quad 00111111 \rightarrow 1087 \\
 \hline
 01011110 \quad 11101001 \\
 + 00000000 \quad 00001101 \rightarrow 13 \\
 \hline
 01011110 \quad 11110110 \\
 + 00000000 \quad 00001111 \rightarrow 15 \\
 \hline
 01011111 \quad 00000101
 \end{array}$$

写不下了写左边



UDP校验和的计算

- 1.把所有数据分成16bit的片段
- 2.依次求和，如果最高位有进位，就挪到最低位进行一次+1操作
- 3.最后得到的结果取反码

$$\begin{array}{r}
 01011111 \quad 00000101 \\
 + 00000000 \quad 00000000 \rightarrow 0(\text{校验和}) \\
 \hline
 01011111 \quad 00000101 \\
 + 01010100 \quad 01000101 \rightarrow \text{数据} \\
 \hline
 10110011 \quad 01001010
 \end{array}$$

$$\begin{array}{r}
 01010011 \quad 01010100 \rightarrow \text{数据} \\
 01001001 \quad 01001110 \rightarrow \text{数据} \\
 01000111 \quad 00000000 \rightarrow \text{数据}
 \end{array}$$

$$\begin{array}{r}
 10011001 \quad 00010011 \rightarrow 153.19 \\
 + 00001000 \quad 01101000 \rightarrow 8.104 \\
 \hline
 10100001 \quad 01111011 \\
 + 10101011 \quad 00000011 \rightarrow 171.3 \\
 \hline
 01001100 \quad 01111110 \\
 + \quad \quad \quad 1 \\
 \hline
 01001100 \quad 01111111 \\
 + 00001110 \quad 00001011 \rightarrow 14.11 \\
 \hline
 01011010 \quad 10001010 \\
 + 00000000 \quad 00010001 \rightarrow 0和17 \\
 \hline
 01011010 \quad 10011011 \\
 + 00000000 \quad 00001111 \rightarrow 15 \\
 \hline
 01011010 \quad 10101010 \\
 + 00000100 \quad 00111111 \rightarrow 1087 \\
 \hline
 01011110 \quad 11101001 \\
 + 00000000 \quad 00001101 \rightarrow 13 \\
 \hline
 01011110 \quad 11110110 \\
 + 00000000 \quad 00001111 \rightarrow 15 \\
 \hline
 01011111 \quad 00000101
 \end{array}$$

写不下了写左边



UDP检验和的计算

- 1.把所有数据分成16bit的片段
- 2.依次求和，如果最高位有进位，就挪到最低位进行一次+1操作
- 3.最后得到的结果取反码

	01011111	00000101	
+	00000000	00000000	→ 0(检验和)
+	01011111	00000101	
	01010100	01000101	→ 数据
+	10110011	01001010	
	01010011	01010100	→ 数据
	1 00000110	10011110	

01001001	01001110	→ 数据
01000111	00000000	→ 数据

10011001	00010011	→ 153.19
+	00001000	01101000 → 8.104
10100001	01111011	
+	10101011	00000011 → 171.3
01001100	01111110	
+		1
01001100	01111111	
+	00001110	00001011 → 14.11
01011010	10001010	
+	00000000	00010001 → 0和17
01011010	10011011	
+	00000000	00001111 → 15
01011010	10101010	
+	00000100	00111111 → 1087
01011110	11101001	
+	00000000	00001101 → 13
01011110	11110110	
+	00000000	00001111 → 15
01011111	00000101	

写不下了写左边



UDP检验和的计算

- 1.把所有数据分成16bit的片段
- 2.依次求和，如果最高位有进位，就挪到最低位进行一次+1操作
- 3.最后得到的结果取反码

	01011111	00000101	
+	00000000	00000000	→ 0(检验和)
+	01011111	00000101	
+	01010100	01000101	→ 数据
+	10110011	01001010	→ 数据
	01010011	01010100	
	00000110	10011110	
+		1	
	00000110	10011111	

01001001	01001110	→ 数据
01000111	00000000	→ 数据

10011001	00010011	→ 153.19
+	00001000	01101000 → 8.104
10100001	01111011	
+	10101011	00000011 → 171.3
01001100	01111110	
+		1
01001100	01111111	
+	00001110	00001011 → 14.11
01011010	10001010	
+	00000000	00010001 → 0和17
01011010	10011011	
+	00000000	00001111 → 15
01011010	10101010	
+	00000100	00111111 → 1087
01011110	11101001	
+	00000000	00001101 → 13
01011110	11110110	
+	00000000	00001111 → 15
01011111	00000101	

写不下了写左边



UDP校验和的计算

- 1.把所有数据分成16bit的片段
- 2.依次求和，如果最高位有进位，就挪到最低位进行一次+1操作
- 3.最后得到的结果取反码

	01011111	00000101	
+	00000000	00000000	→ 0(校验和)
+	01011111	00000101	
+	01010100	01000101	→ 数据
+	10110011	01001010	→ 数据
	01010011	01010100	
	00000110	10011110	
+		1	
	00000110	10011111	
+	01001001	01001110	→ 数据
	01001111	11101101	

01000111 00000000 → 数据

10011001	00010011	→ 153.19
+	00001000	01101000 → 8.104
10100001	01111011	
+	10101011	00000011 → 171.3
01001100	01111110	
+		1
01001100	01111111	
+	00001110	00001011 → 14.11
01011010	10001010	
+	00000000	00010001 → 0和17
01011010	10011011	
+	00000000	00001111 → 15
01011010	10101010	
+	00000100	00111111 → 1087
01011110	11101001	
+	00000000	00001101 → 13
01011110	11110110	
+	00000000	00001111 → 15
01011111	00000101	

写不下了写左边



UDP校验和的计算

- 1.把所有数据分成16bit的片段
- 2.依次求和，如果最高位有进位，就挪到最低位进行一次+1操作
- 3.最后得到的结果取反码

01011111	00000101	
+ 00000000	00000000	→ 0(校验和)
01011111	00000101	
+ 01010100	01000101	→ 数据
10110011	01001010	
+ 01010011	01010100	→ 数据
00000110	10011110	
+ 00000000	1	
00000110	10011111	
+ 01001001	01001110	→ 数据
01001111	11101101	
+ 01000111	00000000	→ 数据
10010100	11101101	

10011001	00010011	→ 153.19
+ 00001000	01101000	→ 8.104
10100001	01111011	
+ 10101011	00000011	→ 171.3
01001100	01111110	
+ 00000000	1	
01001100	01111111	
+ 00001110	00001011	→ 14.11
01011010	10001010	
+ 00000000	00010001	→ 0和17
01011010	10011011	
+ 00000000	00001111	→ 15
01011010	10101010	
+ 00000100	00111111	→ 1087
01011110	11101001	
+ 00000000	00001101	→ 13
01011110	11110110	
+ 00000000	00001111	→ 15
01011111	00000101	

写不下了写左边



UDP校验和的计算

1.把所有数据分成16bit的片段

2.依次求和，如果最高位有进位，就挪到最低位进行一次+1操作

3.最后得到的结果取反码

	01011111	00000101	
+	00000000	00000000	→ 0(校验和)
	01011111	00000101	
+	01010100	01000101	→ 数据
	10110011	01001010	
+	01010011	01010100	→ 数据
	00000110	10011110	
+		1	
	00000110	10011111	
+	01001001	01001110	→ 数据
	01001111	11101101	
+	01000111	00000000	→ 数据
	10010100	11101101	
	01101011	00010010	

全部取反

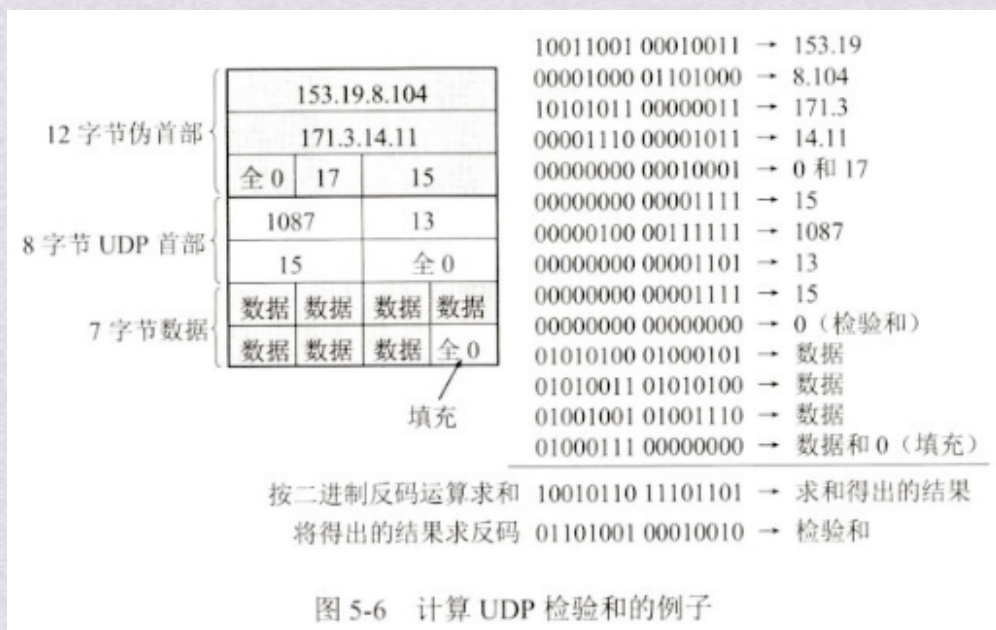
10011001	00010011	→ 153.19
+	00001000	01101000 → 8.104
10100001	01111011	
+	10101011	00000011 → 171.3
01001100	01111110	
+		1
01001100	01111111	
+	00001110	00001011 → 14.11
01011010	10001010	
+	00000000	00010001 → 0和17
01011010	10011011	
+	00000000	00001111 → 15
01011010	10101010	
+	00000100	00111111 → 1087
01011110	11101001	
+	00000000	00001101 → 13
01011110	11110110	
+	00000000	00001111 → 15
01011111	00000101	

写不下了写左边



UDP检验和的计算

- 1.把所有数据分成16bit的片段
- 2.依次求和，如果最高位有进位，就挪到最低位进行一次+1操作
- 3.最后得到的结果取反码



全部取反

10011001 00010011 → 153.19
 + 00001000 01101000 → 8.104

 10100001 01111011
 + 10101011 00000011 → 171.3

 01001100 01111110
 + 1

 01001100 01111111
 + 00001110 00001011 → 14.11

 01011010 10001010
 + 00000000 00010001 → 0和17

 01011010 10011011
 + 00000000 00001111 → 15

 01011010 10101010
 + 00000100 00111111 → 1087

 01011110 11101001
 + 00000000 00001101 → 13

 01011110 11110110
 + 00000000 00001111 → 15

 01011111 00000101

写不下了写左边